Lesson 7 · Token bucket and leaky bucket

Network Modeling · 20 October 2026 · Integrated notes

These notes follow the lecture: a token bucket recap, the two-rate three-color policer, the leaky bucket, the sawtooth exercise, and shaper sizing with burst, grace-period and delay requirements. The professor's explanations and the handwritten notes are the main sources; the book chapter completes the same topics.

1. Checking traffic versus changing traffic

A **policer**, in the model presented by the professor, checks whether a packet respects the traffic contract and **marks** it. Other modules decide whether to forward it with lower priority, treat it as best effort, or drop it.

A **shaper** changes the outgoing traffic: it can hold packets in a queue and release them more slowly. If its finite buffer fills up, excess traffic is lost.

> **Basic idea.** Tokens are credits for transmission; buffers hold data waiting to be transmitted. These are different things.

Notation and units

Symbol	Meaning
ρ	Token generation rate, in tokens per second
b and $z(t)$	Token bucket capacity and available tokens
r	Maximum output rate of the **leaky bucket**
B and $q(t)$	Data buffer capacity and current queued data
$G(t), G_{\text{out}}(t)$	Input and output rates

The professor also uses ρ for the leaky bucket rate. Here we call it r to distinguish it from token generation. We assume **1 token** = **permission to transmit 1 byte**. Thus, ρ numerically corresponds to a rate in bytes/s. When an exercise uses bits/s, we also express buffer capacity in bits.

One-rate two-color token bucket

The bucket receives tokens at rate ρ and holds at most b. For a packet of W bytes:

- If $z(t) \geq W$, the filter consumes W tokens and marks the packet **conformant**.
- Otherwise, it marks the packet **exceed** and **consumes no tokens**.

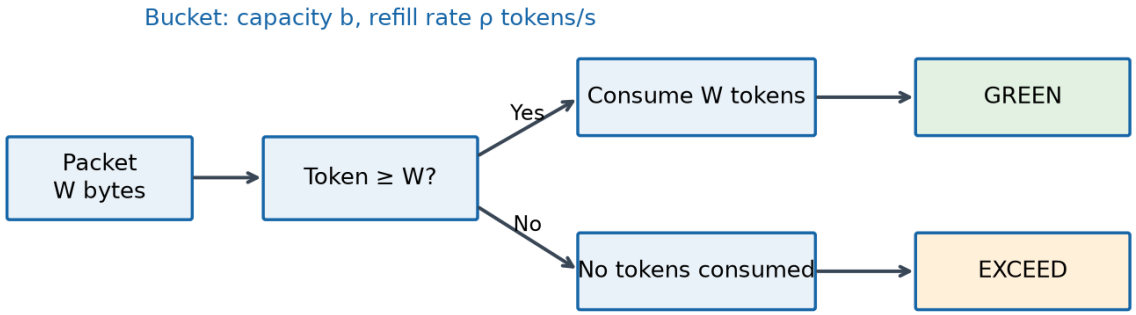


Figure 1 - Decision process of the one-rate policer.

Reading the diagram. The central question concerns credits, not free buffer space. The negative branch assigns a mark; it does not itself require a drop. In the contract, **CIR** (*Committed Information Rate*) is the agreed rate, and **CBS** (*Committed Burst Size*) is the credit capacity, corresponding to b in the minimum configuration.

2. Credits, bursts and the two-rate policer

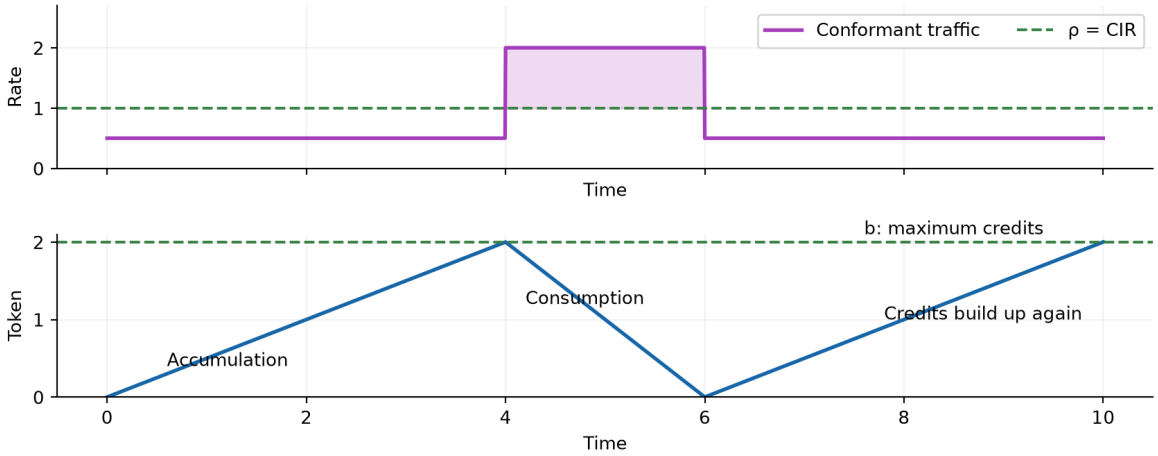


Figure 2 - Conformant traffic and available credits, in illustrative units.

What the graphs mean. When the rate is below ρ, tokens accumulate. During a burst, the rate exceeds ρ and the bucket drains: previously saved credits are spent. Once the bucket reaches b, newly generated unused tokens are lost; waiting longer cannot increase the reserve.

The simple token bucket controls the long-term conformant rate and the accumulated excess, but **does not by itself limit the instantaneous peak rate**. Credits can be spent very quickly. Also, b limits the excess above tokens generated in the meantime, not the total bytes transmitted during a burst of nonzero duration.

Two-rate three-color: two checks, three outcomes

The professor introduces a second bucket to distinguish exceeding the committed profile from also violating the peak profile.

Bucket	Refill rate	Capacity	What it checks
Committed	CIR	CBS	Conformant traffic profile
Peak	PIR, with PIR $\geq$ CIR	PBS	Peak profile, with its own burst allowance

The filter checks the **peak bucket first**, then the committed bucket. For a packet of W bytes:

Available credits	Color	Tokens consumed
Peak $< W$	**Red: violating**	None
Peak $\geq W$, committed $< W$	**Yellow: exceed**	W from peak only
Both $\geq W$	**Green: conformant**	W from each bucket

The professor's qualitative example, using a fluid model. Starting with available credits, if the source maintains $\lambda \leq$ CIR, traffic can remain green. If CIR $< \lambda \leq$ PIR, committed credits decrease and yellow traffic appears. If $\lambda >$ PIR, peak credits can also run out and red packets appear.

This does not mean that every packet necessarily has the same color after the first depletion: tokens keep being generated. The depletion order also depends on CBS, PBS and initial levels. The green-yellow-red sequence described in class is an intuitive example, not a rule independent of those parameters.

This is still a **policer**: colors classify packets. A shaper is needed to impose a hard output-rate limit; the peak bucket still allows a finite burst tolerance.

3. Leaky bucket: a queue with a limited output rate

Imagine a bucket with a hole: data arrives, accumulates when necessary, and leaves at a maximum rate r. The bucket is a **data buffer of capacity** B, not a wallet of tokens.

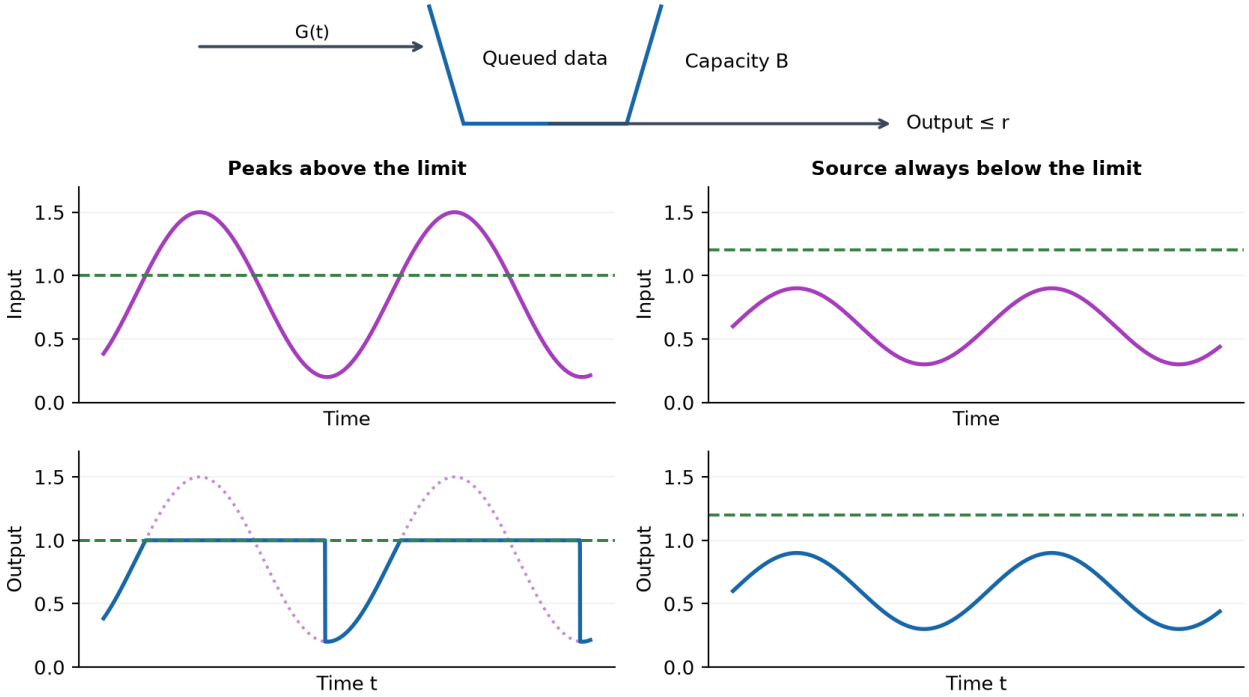


Figure 3 - The bucket analogy and four input/output plots for the leaky bucket.

Right-hand panels. If the buffer is initially empty and the input always stays below r, no queue forms: output equals input.

Left-hand panels. When input exceeds r, output is capped at r and the excess is queued. When input falls below r again, **output stays at r while a backlog remains**. Only after the queue empties does output follow the source again. The dotted purple line helps compare input and output: the peak is spread over a longer time.

The fluid-model rule is:

$$G_{\text{out}}(t) = \begin{cases} \min\{G(t), r\}, & q(t) = 0, \\ r, & q(t) > 0. \end{cases}$$

When $0 < q(t) < B$, the queue changes according to **input minus output**:

$$\frac{dq(t)}{dt} = G(t) - r.$$

If $q(t) = B$ and $G(t) > r$, the buffer cannot grow further: excess data is dropped. A larger buffer absorbs a longer peak, but cannot solve permanent overload.

> **Remember.** "Constant rate" means that the leaky bucket serves at rate r **when it has data to transmit**. It does not create dummy traffic when the queue is empty.

4. Exercise: periodic sawtooth input

The professor proposes a leaky bucket with $r = 1\,\text{Mbit/s}$ and $B = 1\,\text{kB} = 8000\,\text{bits}$, using the decimal kilobyte. The source rate rises linearly from zero towards R and restarts every T seconds:

$$G(t) = \frac{R}{T}\,\text{mod}(t, T) = R\left(\frac{t}{T} - \left\lfloor \frac{t}{T} \right\rfloor\right).$$

Here $\text{mod}(t, T) = t - T\lfloor t/T \rfloor \in [0, T)$, and *floor* rounds down to the nearest integer. The factor $1/T$ is necessary: R **is a rate**, not a slope. The professor corrects this normalization during the lecture.

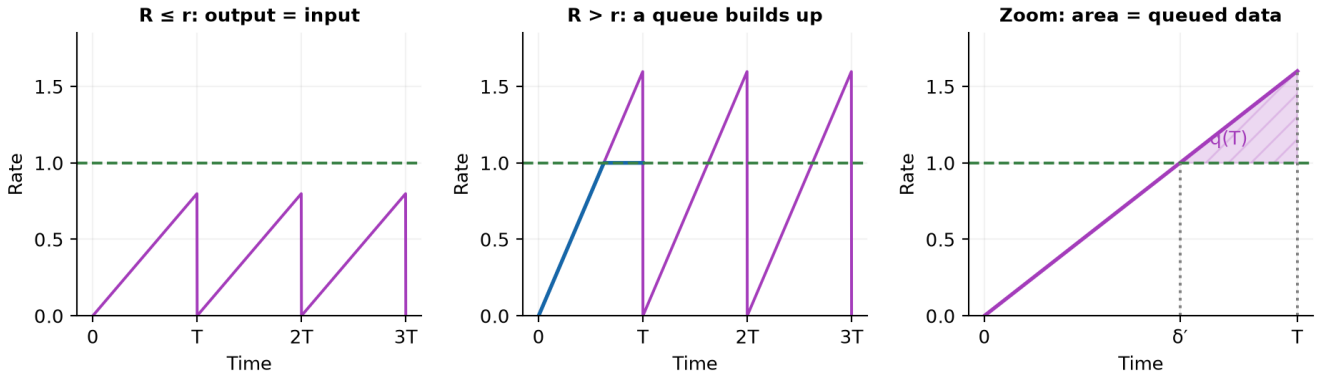


Figure 4 - Sawtooth input below and above the limit, with a close-up of the accumulated data.

Left: $R \leq r$**.** Starting with an empty buffer, $G_{\text{out}}(t) = G(t)$: there is no queue. Output is not constantly R; R is only the peak value approached by the ramp.

Middle: $R > r$**.** During the first period, output follows the ramp until $G(t) = r$, then stays at r:

$$\delta' = \frac{rT}{R}, \qquad \delta = T - \delta' = \frac{R - r}{R}T.$$

Right: close-up of the triangle. The area between input and output measures queued data, because a rate multiplied by time gives an amount of data. At the end of the period:

$$q(T) = \frac{(R - r)\delta}{2} = \boxed{\frac{(R - r)^2 T}{2R}}.$$

This assumes an **initially empty queue and no overflow during the first period**. Therefore, $B \geq q(T)$ is required; otherwise the queue reaches B before T and some excess is dropped.

The entire input triangle has area $RT/2$: this is the data generated in one period. The average rate is therefore $\overline{G} = R/2$. The provisional "RT per period" expression in the handwritten notes needs the factor $1/2$.

5. After input resets to zero: when does the queue empty?

At $t = T$, input restarts from zero, but the buffer still contains $q(T)$ data. Output therefore stays at r. Write $t = T + \varepsilon$, with $0 \le \varepsilon < T$.

While the queue remains positive and there are no losses:

$$q(T+\varepsilon) = q(T) + \underbrace{\int_0^{\varepsilon} \frac{R}{T} u\, du}_{\text{new data}} - \underbrace{r\varepsilon}_{\text{transmitted data}} = q(T) + \frac{R}{2T}\varepsilon^2 - r\varepsilon.$$

Correction to the calculation written in class: we cannot use $G(\varepsilon)\varepsilon$, because the rate increases from zero during the interval. The area is a triangle, so the factor $1/2$ appears again.

To find when the queue empties, set $q(T+\varepsilon) = 0$ and take the **first physically valid root**:

$$\varepsilon_0 = \frac{T}{R}\left[r - \sqrt{r^2 - \frac{2R}{T}q(T)}\right] = \frac{T}{R}\left[r - \sqrt{R(2r-R)}\right].$$

For $r < R < 2r$, this root satisfies $0 < \varepsilon_0 < \delta'$. The parabola's second root does not describe the queue: after it empties, we cannot continue the formula into negative queue levels.

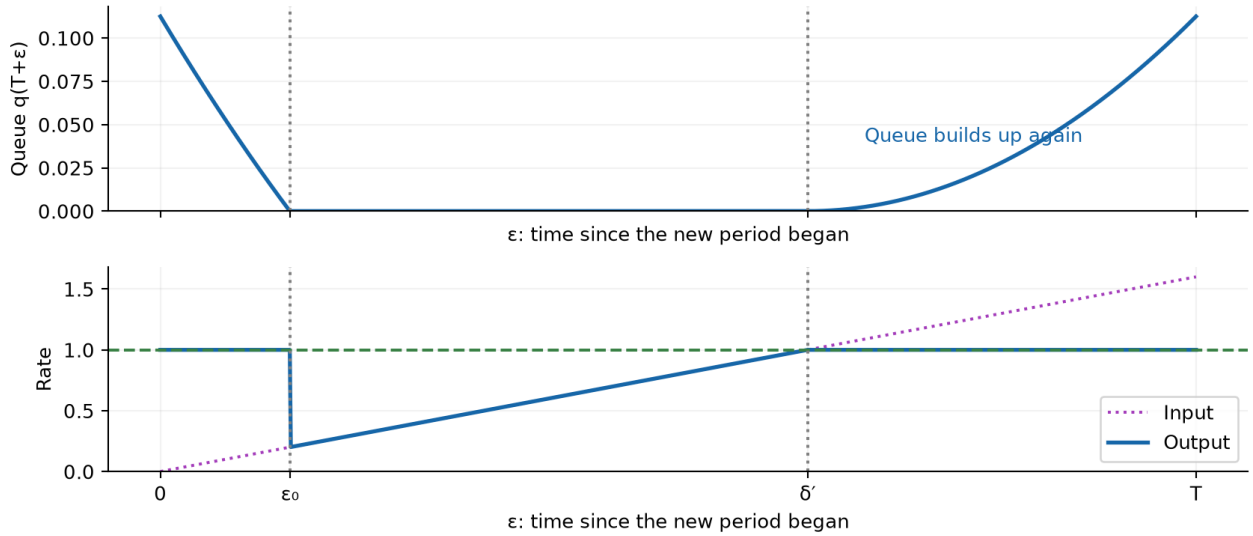


Figure 5 - Queue level and output during the next period, for r < R < 2r.

Upper graph. The queue decreases along a parabola until ε_0, stays at zero until $\delta' = rT/R$, and grows again when the new ramp exceeds r. The handwritten drawing was schematic: the zero-level segment does not continue to the end of the period.

Lower graph. Output equals r until ε_0. When the backlog runs out, output drops to the source's current rate, follows the ramp, and returns to r when input reaches the limit. This explains the downward step drawn by the professor.

6. Stability and losses: completing the exercise

The decisive comparison is between the average input rate $R/2$ and the service rate r:

Condition	Behavior with an initially empty buffer
$R \leq r$	No queue; output equals input
$r < R < 2r$	Queue empties each cycle; no loss if $B \geq q(T)$
$R = 2r$	Boundary case: queue touches zero at $\varepsilon_0 = T/2$ and immediately grows again
$R > 2r$	An unlimited buffer accumulates data across cycles; a finite buffer eventually drops data

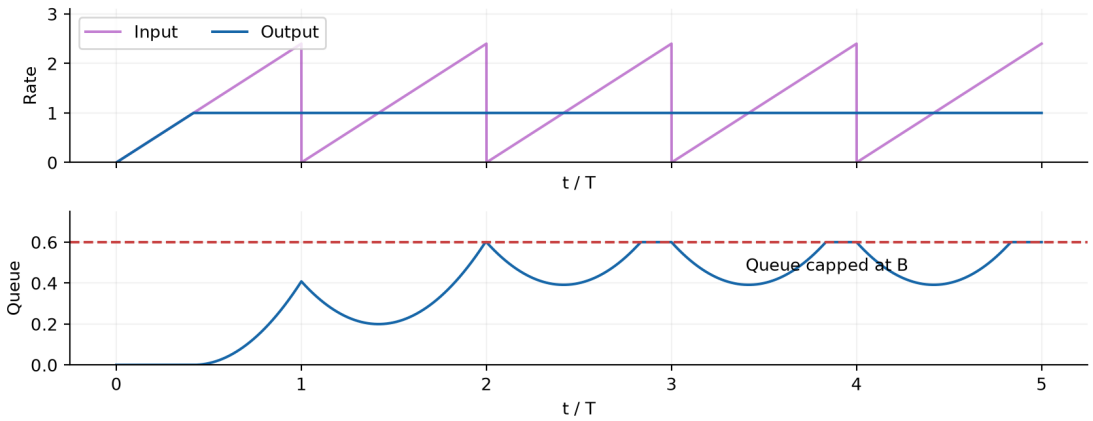


Figure 6 - Periodic overload with a finite buffer.

Top: input keeps oscillating; in this example, after the transient, enough backlog always remains to keep output at r. **Bottom:** the buffer reaches its limit B, then partly drains each cycle. Thus, the note "$q(t) \to B$" should not be taken literally: with periodic input, the queue can oscillate even when losses occur. A very small buffer may even empty, temporarily letting output fall below r.

The professor's simpler example: a CBR source

With constant input $\lambda = 1.5r$, an initially empty queue grows at rate $\lambda - r = 0.5r$. It becomes full after:

$$t_{\text{full}} = \frac{B}{\lambda - r}.$$

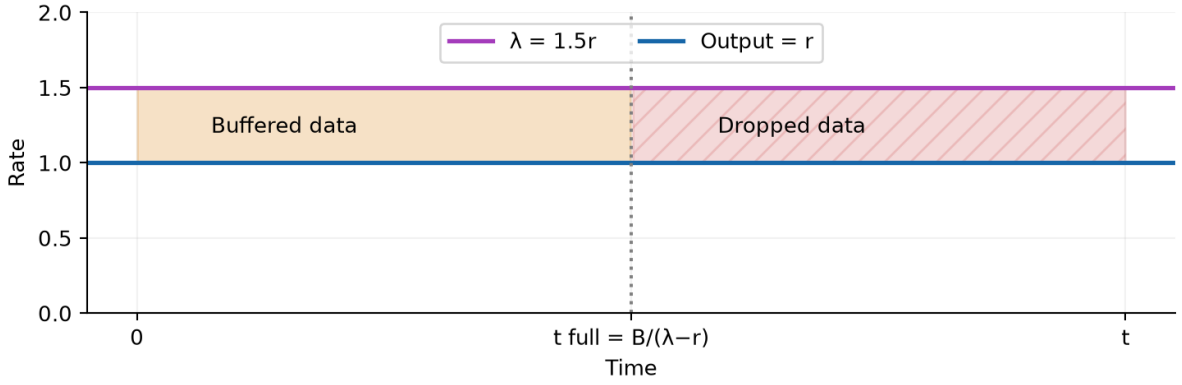


Figure 7 - Excess data is first buffered, then dropped, with constant input.

Meaning of the areas. Before t_{full}, the area between λ and r fills the buffer. Afterwards, the same excess is lost. Output always equals r; the steady-state drop rate is $0.5r$, or **one third of the input**. Unlike the sawtooth case, the buffer stays full because input never decreases.

7. Token bucket as a shaper: burst and grace period

The second part of the lecture combines a token bucket with data buffers and an output link. The source specifies an **SLA** (*Service Level Agreement*), describing its service requirements:

Parameter	Source requirement
λ_a	Long-term average rate
$\lambda_p > \lambda_a$	Peak rate
T_b	Burst duration to be forwarded through the TBF at peak rate
T_g	Additional time at peak rate tolerated without loss: the **grace period**
d_{SLA}	Maximum acceptable delay for data that is not dropped

The **burst** uses available credits and passes through the TBF at λ_p. During the **grace period**, credits have run out: the source may still generate at λ_p, but the TBF forwards at ρ. The difference must wait in a queue.

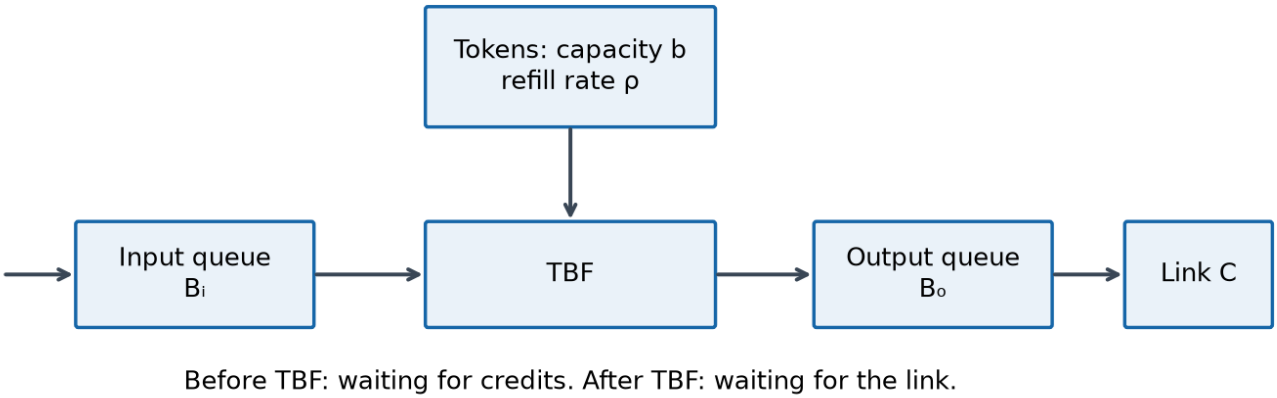


Figure 8 - Token bucket, input buffer, output buffer and link.

Reading the diagram from left to right.

- Buffer B_i holds data waiting for tokens.
- Bucket b holds only credits; the filter consumes them to authorize data forwarding.
- Buffer B_o holds data that has passed the filter but is waiting for link transmission.
- The link transmits at a maximum rate C.

The combination **output buffer + link** behaves like a leaky bucket of rate C. This explains the combined design: the TBF allows credit-limited bursts; the link imposes the final physical limit. If $C < \lambda_p$, a burst passes through the TBF at peak rate but **does not leave the link at peak rate**: it queues in B_o.

Assumptions behind the professor's calculations

We use a **fluid model**: data is a continuous stream, without packet boundaries. At the start of the burst, the token bucket is full and both data buffers are empty. Queues are FIFO, the source does not exceed λ_p, and TBF forwarding in the model is limited to that peak. We study the worst case where the source stays at peak rate throughout the burst and grace period.

The peak limit is an additional assumption or configuration of this system: a simple bucket with only ρ and b does not enforce it by itself. The calculated delays are queuing delays in the model, excluding propagation and packet serialization details.

8. Sizing tokens and buffers using rate balances

The common rule is: **accumulated amount = rate difference × duration**. For a minimum configuration, choose $\rho = \lambda_a$; generally, $\rho \geq \lambda_a$ is needed. We consider $\rho < \lambda_p$ and a link with $C > \rho$, so it can drain the backlog while carrying post-burst traffic.

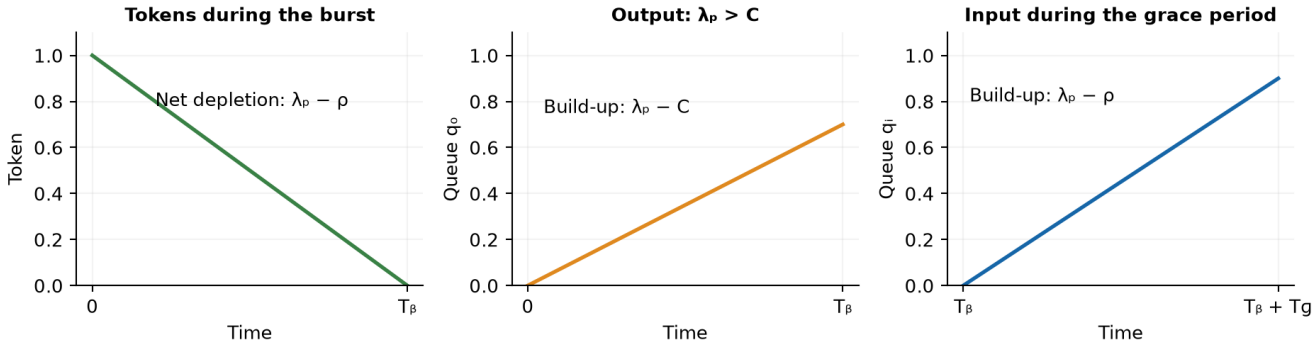


Figure 9 - Three balances: token depletion, output queue growth, and input queue growth.

Credits needed for the burst

Left panel. Tokens leave at rate λ_p, but keep arriving at rate ρ. The level therefore decreases at $\lambda_p - \rho$, not at λ_p:

$$T_{\text{depletion}} = \frac{b}{\lambda_p - \rho}.$$

To allow at least T_b, we need $b \geq (\lambda_p - \rho)T_b$. To make credits run out exactly at the end of the agreed burst, choose:

$$\boxed{b = (\lambda_p - \rho)T_b}.$$

The following formulas use this choice. Increasing b also increases the effective burst duration that must be considered when sizing the output buffer.

Output buffer

Middle panel. If $\rho < C < \lambda_p$, then $\lambda_p T_b$ bytes arrive and CT_b bytes depart during T_b. Therefore:

$$\boxed{B_o \geq (\lambda_p - C)T_b}.$$

If $C \geq \lambda_p$, no output backlog builds up and $B_o = 0$ is sufficient in the fluid model. In practice, some memory is still needed to handle packets. Buying a link much faster than the peak avoids this queue but may waste capacity: this is the trade-off highlighted by the professor.

Input buffer and grace period

Right panel. After token depletion, λ_p bytes/s arrive, but only ρ pass through. To tolerate a grace period T_g:

$$\boxed{B_i \geq (\lambda_p - \rho)T_g}.$$

With equality, the buffer fills at $t_p = T_b + T_g$. If the source stays at peak rate beyond that time, excess traffic is dropped. Filling a data buffer differs from exhausting tokens: the former causes losses; the latter reduces the forwarding rate.

9. Maximum delay with a fast link: $C \geq \lambda_p$

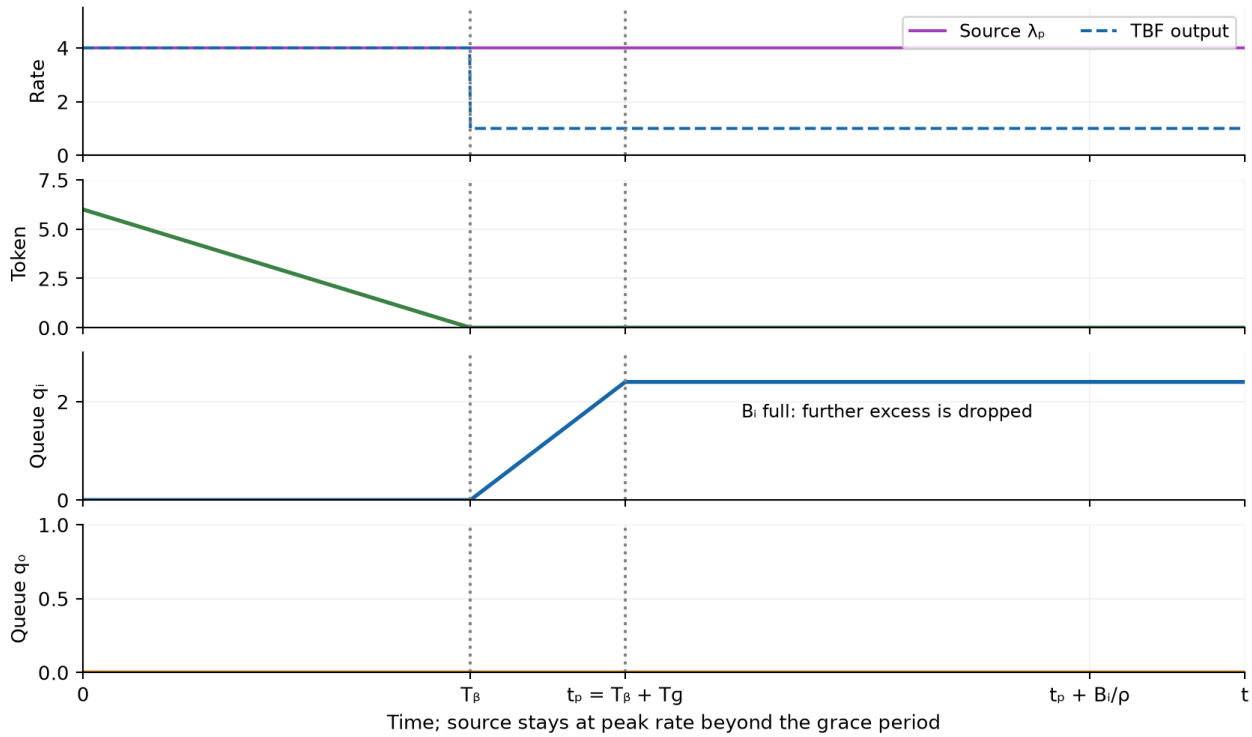


Figure 10 - Rates, tokens, input queue and output queue with a fast link.

These are the graphs described by the professor for the blank space on the last handwritten page. Here the source stays at peak rate beyond the grace period to show saturation and drops.

First graph: rates. Until T_b, the TBF forwards at peak rate. After T_b, tokens are exhausted and it forwards only at ρ.

Second graph: tokens. The reserve decreases linearly from b to zero at rate $\lambda_p - \rho$. Afterwards, newly generated tokens are consumed immediately, so the level stays at zero.

Third graph: input. The queue is empty during the burst; it grows during the grace period at rate $\lambda_p - \rho$ and reaches B_i at t_p. If the peak continues, the buffer remains full and further excess is dropped.

Fourth graph: output. This queue is always empty because the link can transmit at least as fast as the TBF supplies data.

The worst-case data is accepted at the tail of a full input queue. Approximately B_i bytes are ahead of it, served at rate ρ:

$$\boxed{d_{\max} = \frac{B_i}{\rho}}.$$

To meet the delay requirement and guarantee the grace period, both bounds must hold:

$$(\lambda_p - \rho)T_g \leq B_i \leq \rho d_{\text{SLA}}.$$

Greater tolerance therefore needs more memory, but may also increase the maximum wait. If the source drops below ρ, the queue starts draining. Tokens accumulate again **after the backlog has cleared**: while queued data remains, all newly generated tokens are still needed to forward it.

10. Two queues: $\rho < C < \lambda_p$

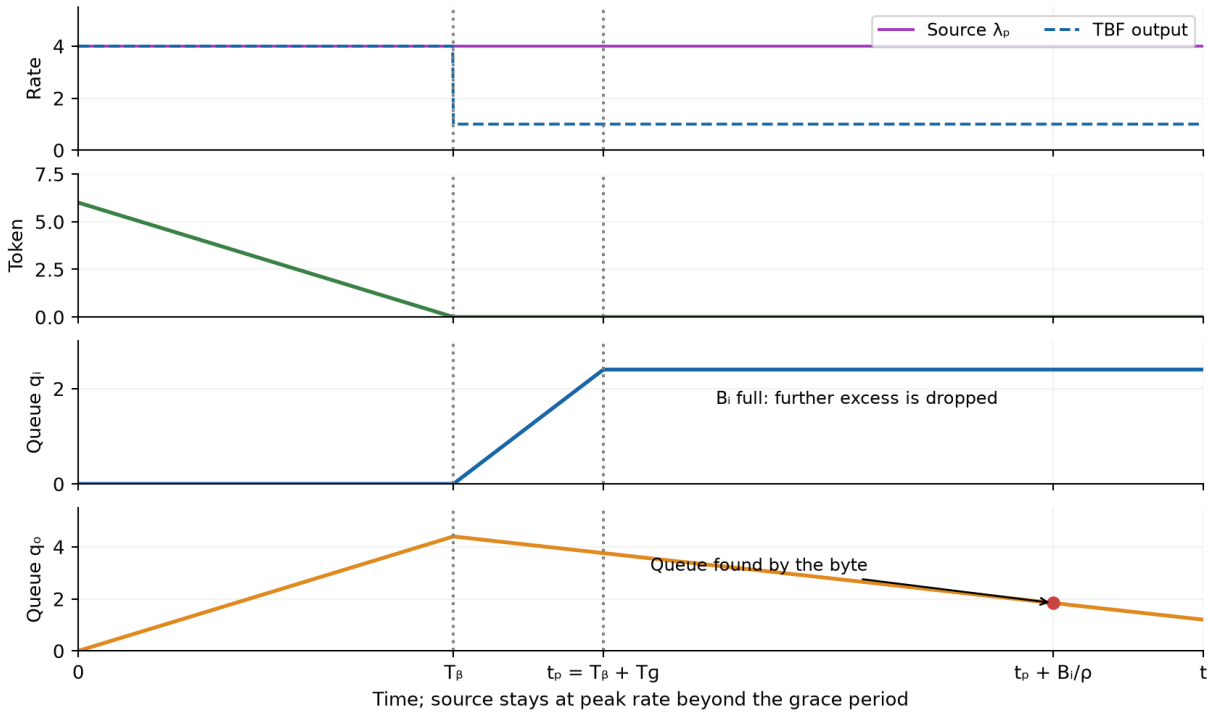


Figure 11 - During the grace period, the input queue grows while the output queue drains.

Rates and tokens. These behave as before: forwarding drops from λ_p to ρ when credits run out.

Input queue. It is empty during the burst, then grows during the grace period until it reaches B_i.

Output queue. During the burst, it grows at rate $\lambda_p - C$ and reaches its maximum

$$Q_o = (\lambda_p - C)T_b$$

at $t = T_b$. Here Q_o is the actual maximum backlog: the physical capacity B_o may be larger. After T_b, the TBF supplies $\rho < C$, so the queue drains at rate $C - \rho$:

$$q_o(t) = \max\{0, Q_o - (C - \rho)(t - T_b)\}, \qquad t \geq T_b,$$

as long as the TBF keeps forwarding at ρ. The source stays at peak rate in the illustrated example, so this condition holds.

The central point in the professor's explanation

Data generated at time t first waits in the input queue for $d_i(t)$. It therefore reaches the output queue at time $t + d_i(t)$. Its total delay is:

$$\boxed{d(t) = d_i(t) + \frac{q_o(t + d_i(t))}{C}}.$$

Do not add the two queues as photographed at the same instant: while data waits at the input, the output queue is already draining. The red point marks the backlog found **when that data actually reaches the output queue**. With different parameters, the output queue could already be empty then.

11. Completing the calculation and checking the SLA

Set $t_p = T_b + T_g$ and choose $B_i = (\lambda_p - \rho)T_g$. Data accepted at the end of the grace period finds the input queue full and waits:

$$d_i = \frac{B_i}{\rho}.$$

During the $T_g + d_i$ elapsed since token depletion, the output queue has decreased. The maximum delay is therefore:

$$\boxed{d_{\max} = \frac{B_i}{\rho} + \frac{\max\{0, Q_o - (C-\rho)(T_g + B_i/\rho)\}}{C}}.$$

This completes the procedure explained in the final minutes of the lecture. The chapter gives the same result in a compact form:

$$\boxed{d_{\max} = \max\left\{\frac{B_i}{\rho},\ \frac{Q_o}{C} + \frac{\lambda_p - C}{C}T_g\right\}}, \qquad \rho < C < \lambda_p.$$

Why is the worst case at the end of the grace period? During that period, let $\tau = t - T_b$. While an output backlog remains when the data arrives, $d(t) = Q_o/C + \tau(\lambda_p/C - 1)$, which increases because $\lambda_p > C$. When the output queue is already empty, $d(t) = (\lambda_p - \rho)\tau/\rho$, which also increases. Comparing the two queue sizes is not enough: we must follow the same data through both.

The contract is satisfied only if $d_{\max} \le d_{\text{SLA}}$. For two queues, check both conditions:

$$\frac{B_i}{\rho} \le d_{\text{SLA}}, \qquad \frac{Q_o}{C} + \frac{\lambda_p - C}{C}T_g \le d_{\text{SLA}}.$$

Quick checks to remember

- A **policer marks**; a **shaper delays** and may drop data when its buffer is full.
- A **token bucket saves credits** and allows bursts; a **leaky bucket stores data** and always caps output at r.
- For a queue, **integrating the rate difference gives an area**: a triangular region contributes the factor $1/2$.
- For queues in series, evaluate the second queue **when the data arrives there**, not when it is generated.

Sources and coverage of the drawings

Main sources: all 7 pages of `Lesson 7.pdf`; `20 ott 2026, 08-45.transcript.txt`, from 00:00 to 01:33:54. **Targeted additions:** `4-QoS-TokenFilters-BookChapter.pdf`, §0.2 and the parts of §0.3 covering policers, soft shaping, bursts, buffers and delays (§0.3.0.1-0.3.0.2). The general QoS pipeline, CAC, scheduling and network slicing are excluded because they were not developed in this lecture.

The drawings are recreated and explained: handwritten p. 1 → fig. 1 and the color table; p. 2 → fig. 3-4; p. 3 → fig. 4, including the close-up; p. 4 → fig. 5-6, with corrections; p. 5 → fig. 7-8; p. 6 → fig. 8-9; p. 7 → fig. 8 and 10-11. Figure 2 reconstructs the credit graph described orally. Repeated TBF diagrams are combined in fig. 8 and the three balances in fig. 9.

The provisional sawtooth formulas have been recalculated. The two-queue delay also corrects the chapter's typo labeling the output backlog with the input subscript. The chapter's final inverse formulas for B_i, not developed by the professor and inconsistent in the supplied version, are not used: the checks here follow directly from the delay expression.